

Revisão detalhada

Da modelagem aos dados semiestruturados

Material de apoio aos estudos

Conteúdos

DER e cardinalidades

Modelo relacional

Normalização

Álgebra Relacional

SQL

JSON e documentos

O projeto de banco de dados

As decisões avançam da realidade para a implementação

1

Realidade

processos e
informações

2

Requisitos

dados necessários

3

DER (Diagrama Entidade-Relacionamento)

entidades e regras

4

Modelo relacional

tabelas e chaves

5

SQL

estrutura e
consultas

A modelagem conceitual descreve o domínio. O modelo relacional prepara a implementação no SGBD.

Banco de dados e SGBD

O banco contém os dados. O SGBD administra seu armazenamento e acesso

BANCO DE DADOS

- conjunto organizado de dados relacionados
- representa informações de um domínio
- atende consultas e operações da aplicação

SGBD

- controla usuários e permissões
- aplica restrições de integridade
- executa consultas e controla transações

Exemplos de SGBD: Oracle Database, PostgreSQL, MySQL e SQL Server.

Elementos do DER

O diagrama representa objetos, características e associações do domínio

Entidade

Objeto relevante

PILOTO, VOO

Atributo

Característica

nome, destino

Identificador

Distingue ocorrências

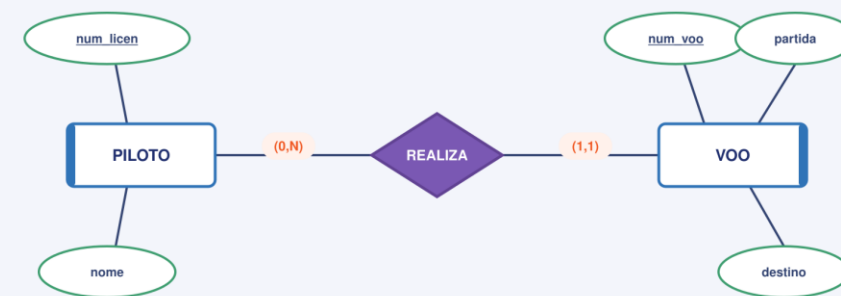
num_licen

Relacionamento

Associação entre entidades

PILOTO realiza VOO

Exemplo de DER — Companhia Aérea



Um piloto pode realizar nenhum ou vários voos; cada voo é realizado por exatamente um piloto.

No DER, o relacionamento recebe nome verbal: realiza, possui, cursa, pertence.

Como interpretar as cardinalidades

No par (mínimo, máximo), cada valor responde a uma pergunta diferente

PILOTO

(0,N)

REALIZA

(1,1)

VOO

PILOTO (0,N)

0

Pode estar cadastrado sem realizar voos

N

Pode realizar vários voos

Participação opcional e múltipla

VOO (1,1)

1

Deve estar associado a pelo menos um piloto

1

Está associado a no máximo um piloto

Participação obrigatória e única

Primeiro valor: mínimo e obrigatoriedade. Segundo valor: máximo e quantidade.

Desafio 1 - Modelagem conceitual

Situação: clientes encomendam casas de diferentes estilos arquitetônicos

Cada cliente possui CPF e nome. A encomenda registra metragem, número de cômodos, valor pago, data da compra e forma de pagamento. Cada casa possui um estilo arquitetônico identificado por um código.

Identifique

a) As entidades principais.

b) Dois relacionamentos possíveis.

c) Os atributos da encomenda.

d) Uma possível cardinalidade entre cliente e encomenda.

Registre sua proposta. A modelagem pode variar, mas precisa manter coerência com os requisitos.

Gabarito comentado - Desafio 1

Compare o raciocínio usado, além da resposta final

DESAFIO 1

Possível modelagem

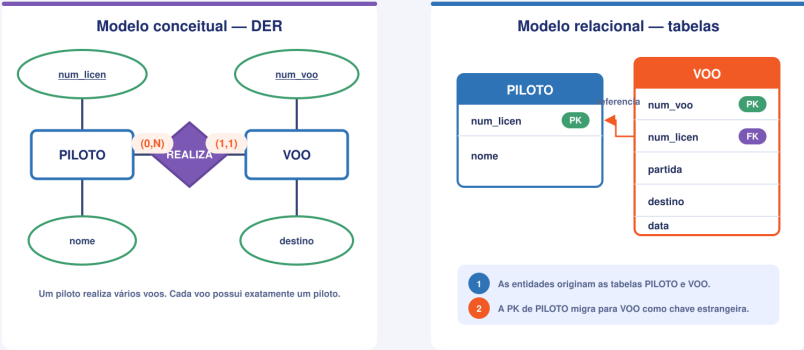
- Entidades: CLIENTE, CASA e ESTILO
- Relacionamentos: CLIENTE encomenda CASA; CASA possui ESTILO
- Atributos da encomenda: metragem, número de cômodos, valor, data e forma de pagamento
- Cardinalidade possível: CLIENTE 1:N ENCOMENDA

ENCOMENDA também pode ser modelada como entidade associativa.

Mapeamento para o modelo relacional

Cada elemento conceitual recebe uma representação lógica

Exemplo de mapeamento 1:N



No relacionamento 1:N, a chave estrangeira fica na tabela correspondente ao lado N.

Entidade

Tabela

PILOTO(num_licen, nome)

Identificador

Chave primária

PK: num_licen

Relacionamento 1:N

FK no lado N

VOO.num_licen referencia PILOTO

Relacionamento N:N

Tabela associativa

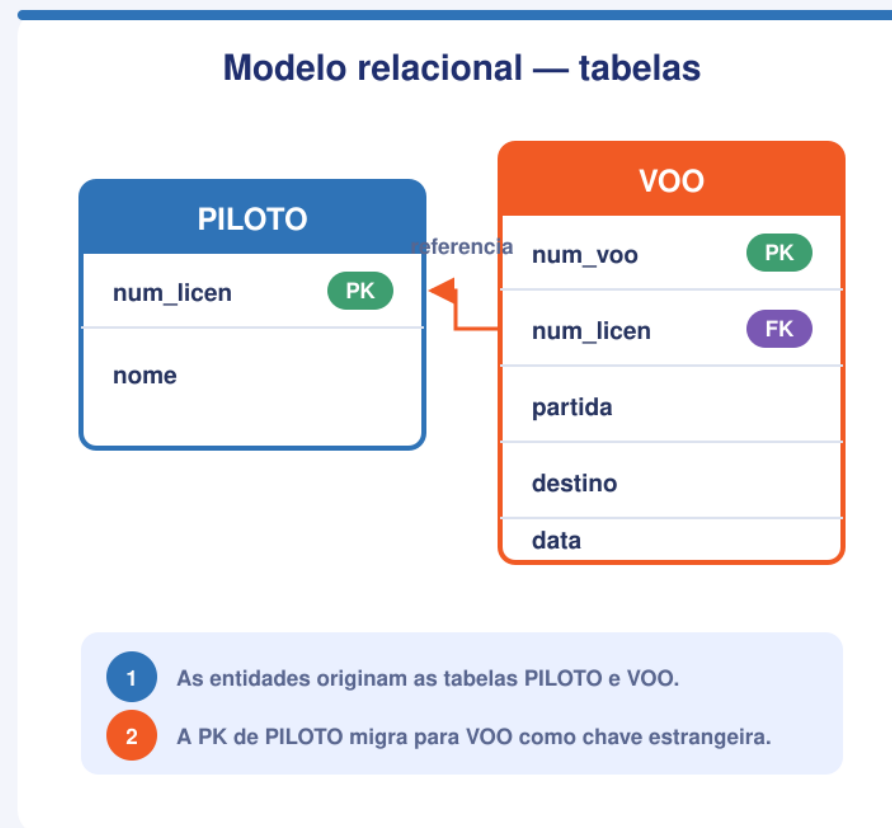
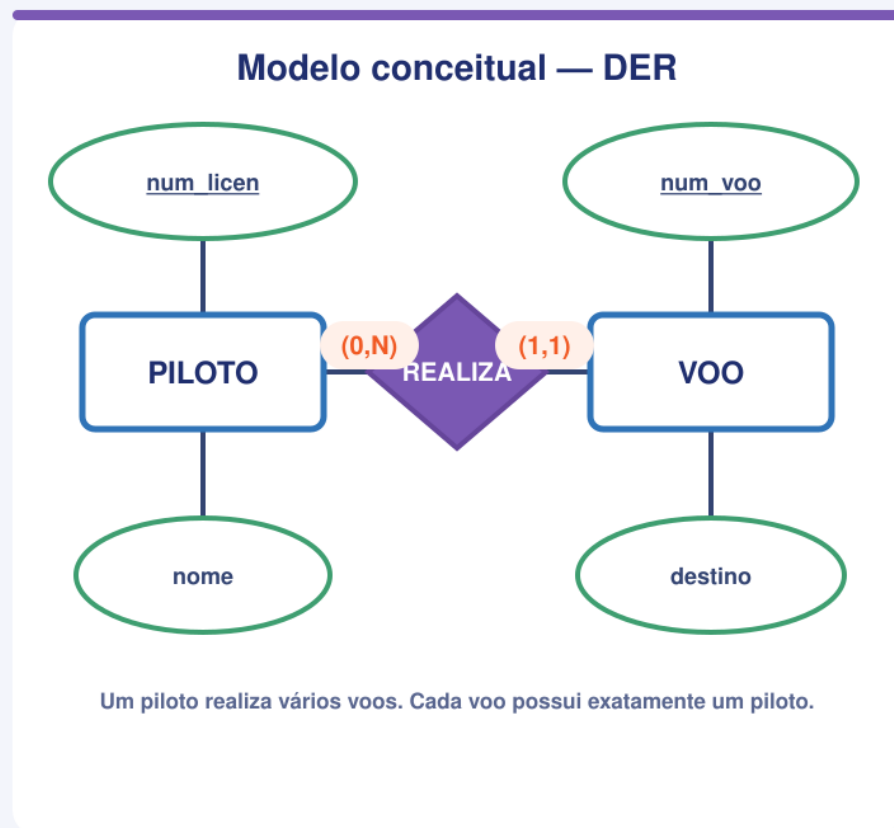
CURSA(id_aluno, id_disciplina, data, nota)

Relacionamentos N:N precisam de uma nova tabela para registrar as associações.

Mapeamento para o modelo relacional

Cada elemento conceitual recebe uma representação lógica

Exemplo de mapeamento 1:N



No relacionamento 1:N, a chave estrangeira fica na tabela correspondente ao lado N.

Chaves e integridade

As restrições impedem estados inválidos no banco de dados

Chave primária

identifica cada linha

não repete e não aceita NULL

NOT NULL

torna o preenchimento obrigatório

nome do piloto, por exemplo

Chave estrangeira

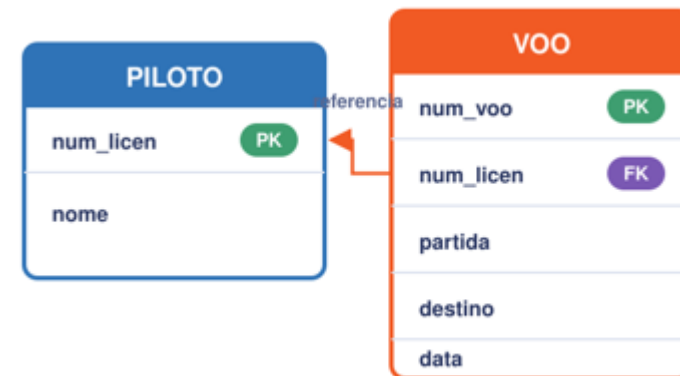
liga tabelas relacionadas

deve referenciar valor existente ou aceitar NULL

UNIQUE

impede valores repetidos

CPF, e-mail ou matrícula



Integridade referencial evita apontar para uma linha inexistente na tabela relacionada.

Por que normalizar?

A normalização reduz redundâncias e anomalias de atualização

Exemplo problemático

```
Livro(cod_livro, título, cod_autor, nome_autor, cod_editora, nome_editora, cidade_editora)
```

Redundância

O nome do autor e da editora se repete em vários livros.

Anomalia de atualização

Alterar a cidade exige atualizar várias linhas.

Anomalia de exclusão

Excluir o último livro pode apagar os dados da editora.

Normalizar significa decompor relações preservando os dados e as dependências necessárias.

Primeira Forma Normal - 1FN

Cada atributo deve conter um valor atômico

ANTES

ALUNO

telefones

Ana

9999, 8888

Uma célula contém dois valores.

DEPOIS

ALUNO

TELEFONE

Ana

9999

Ana

8888

Cada linha registra um telefone.

A 1FN também elimina grupos repetidos. Ela não resolve dependências parciais ou transitivas.

Segunda Forma Normal - 2FN

Todo atributo não-chave deve depender da chave composta inteira

```
MATRICULA(ra, cod_disc, nome_aluno, nome_disc, nota)
PK = (ra, cod_disc)
```

Dependências parciais

```
ra determina nome_aluno
cod_disc determina nome_disc
```

Decomposição

```
ALUNO(ra, nome_aluno)
DISCIPLINA(cod_disc, nome_disc)
MATRICULA(ra, cod_disc, nota)
```

Com chave simples, não existe dependência parcial. Nesse caso, a 2FN é atendida automaticamente.

Terceira Forma Normal - 3FN

Atributos não-chave não devem determinar outros atributos não-chave

```
LIVRO(cod_livro, título, cod_autor, nome_autor)
cod_livro determina cod_autor; cod_autor determina nome_autor
```

Dependência transitiva

cod_livro determina nome_autor por meio de cod_autor.

LIVRO

(cod_livro, título, cod_autor)

AUTOR

(cod_autor, nome_autor)

Resultado: cada fato fica armazenado na relação que representa seu objeto.

Desafio 2 - Normalização

Determine a forma normal mais elevada da relação

```
ALUNO_DISCIPLINA(ra, cod_disc, nome_aluno, nome_disc, nota, cod_prof, nome_prof)
PK = (ra, cod_disc)
```

Dependências informadas

1. ra determina nome_aluno
2. cod_disc determina nome_disc e cod_prof
3. cod_prof determina nome_prof
4. (ra, cod_disc) determina nota

Pergunta: a relação está na 1FN, 2FN ou 3FN? Justifique com as dependências.

Gabarito comentado - Desafio 2

Compare o raciocínio usado, além da resposta final

```
ALUNO_DISCIPLINA(ra, cod_disc, nome_aluno, nome_disc, nota, cod_prof, nome_prof)
PK = (ra, cod_disc)
```

DESAFIO 2

Forma normal mais elevada: 1FN

- nome_aluno depende apenas de ra
- nome_disc e cod_prof dependem apenas de cod_disc
- essas dependências são parciais
- nome_prof depende transitivamente de cod_prof

Como a relação falha na 2FN, ela permanece na 1FN.

Operações da Álgebra Relacional

Cada operação recebe uma ou mais relações e produz uma nova relação

Operações unárias



Seleção

Filtra as linhas por uma condição.

`destino = 'Curitiba'`



Projeção

Escolhe os atributos do resultado.

`nome, destino`

Seleção atua nas linhas. Projeção atua nas colunas.

Operações de conjunto



União

Reúne as tuplas das duas relações.

$R \cup S$



Interseção

Mantém as tuplas presentes nas duas.

$R \cap S$



Diferença

Mantém as tuplas de R ausentes em S.

$R - S$

Combinação



Produto cartesiano

Forma todos os pares possíveis entre R e S.

$R \times S$

Se R possui m tuplas e S possui n, o resultado pode ter $m \times n$ tuplas.

Uma junção equivale a combinar relações e manter apenas os pares que atendem ao vínculo.

União, interseção e diferença exigem relações compatíveis: mesma quantidade de atributos e domínios correspondentes.

Junções na Álgebra Relacional

A junção combina tuplas relacionadas por uma condição ou por atributos comuns



Junção condicional

Usa uma condição explícita para relacionar as tuplas.

```
Piloto.num_licen =  
Voo.num_licen
```



Junção natural

Usa atributos com o mesmo nome e mantém uma única cópia da coluna comum.

```
Piloto ∞ Voo
```



Junção externa

Mantém também as tuplas sem correspondência em uma ou nas duas relações.

```
LEFT, RIGHT e FULL
```

Variações da junção externa

LEFT: preserva R

RIGHT: preserva S

FULL: preserva R e S

Correspondência com SQL

σ WHERE

π SELECT

$\bowtie$ JOIN

$\cup$ UNION

$\cap$
INTERSECT

$-$
MINUS

A junção natural exige cuidado: atributos com o mesmo nome são usados automaticamente como vínculo.

Construção de uma consulta algébrica

Pergunta: quais pilotos realizaram voos com destino a Curitiba?

Piloto(num_licen, nome)

Voo(num_voo, destino, num_licen)

1. Junção

Piloto $\bowtie$ Voo

reúne atributos das duas relações

2. Seleção

σ destino='Curitiba'

mantém somente os voos desejados

3. Projeção

π nome

retorna apenas o atributo solicitado

Expressão: π nome (σ destino='Curitiba' (Piloto $\bowtie$ Voo))

Desafio 3 - Álgebra Relacional

Obtenha os destinos dos voos realizados pelo piloto Pedro Costa

Piloto(num_licen, nome) Voo(num_voo, partida, destino, num_licen)

Organize as operações

- 1 Localizar o piloto pelo nome.
- 2 Combinar Piloto e Voo pelo número da licença.
- 3 Retornar somente o atributo destino.

Escreva a expressão completa usando seleção, junção e projeção.

Gabarito comentado - Desafio 3

As operações precisam manter disponíveis os atributos necessários

Piloto(num_licen, nome)

Voo(num_voo, partida, destino, num_licen)

DESAFIO 3

Álgebra Relacional

```
π destino (  
  σ nome='Pedro Costa' (  
    Piloto ⋈ Voo  
  )  
)
```

Também é possível selecionar Pedro Costa antes da junção.

Grupos de comandos SQL

A linguagem define estruturas, manipula registros e recupera informações

DDL (Linguagem de Definição de Dados)

estrutura

CREATE, ALTER, DROP

DML (Linguagem de Manipulação de Dados)

registros

INSERT, UPDATE, DELETE

DQL (Linguagem de Consulta de Dados)

consultas

SELECT

Restrições de integridade definidas pela DDL

integridade

PRIMARY KEY, FOREIGN KEY, NOT NULL

SQL é declarativa: você informa o resultado desejado e o SGBD escolhe como executar.

Criação das tabelas

As restrições podem ser declaradas junto com a estrutura

```
CREATE TABLE Piloto (  
    num_licen NUMBER PRIMARY KEY,  
    nome VARCHAR2(100) NOT NULL  
);  
  
CREATE TABLE Voo (  
    num_voo VARCHAR2(10) PRIMARY KEY,  
    destino VARCHAR2(80) NOT NULL,  
    num_licen NUMBER,  
    FOREIGN KEY (num_licen) REFERENCES Piloto(num_licen)  
);
```

Leitura do código

- Piloto deve existir antes de Voo
- a PK identifica cada registro
- NOT NULL exige um valor
- a FK cria a ligação entre as tabelas
- o tipo da FK deve ser compatível

Manipulação segura dos dados

INSERT inclui, UPDATE altera e DELETE remove registros

```
INSERT INTO Piloto (num_licen, nome)
VALUES (123456, 'João Silva');
```

```
UPDATE Piloto
SET nome = 'João da Silva'
WHERE num_licen = 123456;
```

```
DELETE FROM Piloto
WHERE num_licen = 123456;
```

Cuidado

UPDATE ou DELETE sem WHERE pode atingir todas as linhas da tabela.

Transação

COMMIT confirma as alterações. ROLLBACK desfaz alterações ainda não confirmadas.

Estrutura de uma consulta SELECT

A ordem escrita difere da ordem lógica de processamento

```
SELECT coluna, função_agregada  
FROM tabela  
JOIN outra_tabela ON condição  
WHERE condição_das_linhas  
GROUP BY coluna  
HAVING condição_dos_grupos  
ORDER BY coluna;
```

Ordem lógica simplificada

1. FROM e JOIN
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

WHERE filtra linhas antes do agrupamento. HAVING filtra os grupos formados.

Junções e agregações

JOIN combina tabelas. GROUP BY produz resumos por categoria

JUNÇÕES

INNER JOIN

Somente linhas com correspondência.

LEFT JOIN

Todas as linhas da esquerda, mesmo sem correspondência.

```
FROM Modelo m
LEFT JOIN Aviao a
  ON a.id_modelo = m.id_modelo
```

AGREGAÇÃO

COUNT, SUM, AVG, MIN, MAX

GROUP BY reúne linhas com o mesmo valor. HAVING aplica uma condição depois da agregação.

```
GROUP BY p.num_licen, p.nome
HAVING COUNT(*) > 1
```

Desafio 4 - SQL

Liste os pilotos que possuem mais de um voo cadastrado

Piloto(num_licen, nome)

Voo(num_voo, destino, num_licen)

```
SELECT _____  
FROM Piloto p  
INNER JOIN Voo v ON _____  
GROUP BY _____  
HAVING _____  
ORDER BY quantidade DESC;
```

Complete SELECT, condição do JOIN, GROUP BY e HAVING.

Gabarito comentado - Desafio 4

As operações precisam manter disponíveis os atributos necessários

Piloto(num_licen, nome)

Voo(num_voo, destino, num_licen)

DESAFIO 4

SQL

```
SELECT p.nome, COUNT(*) AS quantidade
FROM Piloto p
INNER JOIN Voo v
    ON v.num_licen = p.num_licen
GROUP BY p.num_licen, p.nome
HAVING COUNT(*) > 1
ORDER BY quantidade DESC;
```

Dados semiestruturados e JSON

JSON organiza valores em objetos e arrays aninhados

```
{
  "codigo": "R002",
  "cliente": "João Lima",
  "voos": [
    {"numero": "V002", "destino": "Recife"},
    {"numero": "V003", "destino": "Fortaleza"}
  ],
  "bagagem": true
}
```

Conceitos

- objeto: conjunto de pares chave e valor
- array: lista ordenada de valores
- valores: string, number, boolean, null, objeto ou array
- documentos de uma coleção podem variar
- esquema flexível também pode ter validação

Campo ausente e campo com valor null representam situações diferentes.

Quadro de consulta rápida

Use este resumo para revisar antes da prova

DER

entidades, atributos, relacionamentos e cardinalidades

1FN

valores atômicos e sem grupos repetidos

3FN

sem dependência transitiva entre não-chaves

SQL

WHERE filtra linhas; HAVING filtra grupos

Modelo relacional

tabelas, PK, FK e restrições

2FN

sem dependência parcial da chave composta

Álgebra

selecionar, projetar, juntar e combinar relações

JSON

objetos, arrays e estrutura flexível

Explique o raciocínio, confira os atributos disponíveis e revise as chaves.